

Notes on Tensor Concepts, Tucker, and Canonical Decompositions

1 Tensors: Definitions, Core Concepts, and Real-World Examples

What is a tensor? A **tensor** $\mathcal{X} \in \mathbb{R}^{I_1 \times \dots \times I_N}$ is an N -dimensional array (also called an N -way array). A 1D tensor is a *vector*, a 2D tensor is a *matrix*, and tensors with $N \geq 3$ are *higher-order* tensors. The *order* (or *mode/way*) is N , and the *shape* (dimensions) is $(I_1, \dots, I_N)$.

Basic notation An order- N real tensor is written $\mathcal{X} \in \mathbb{R}^{I_1 \times \dots \times I_N}$ with entries $x_{i_1, \dots, i_N}$. We use the following terms:

- **Order (modes/ways):** the number of axes (do not confuse with matrix rank).
- **Fiber:** fixing all but one index yields a vector, e.g., $\mathcal{X}_{:,j,k}$.
- **Slice:** fixing all but two indices yields a matrix, e.g., $\mathcal{X}_{:, :, k}$.
- **Unfolding (matricization):** reshape $\mathcal{X}$ so that one mode forms the rows and all remaining modes (flattened in a chosen, consistent order) form the columns; the mode- n unfolding is denoted $X_{(n)}$.

Fibers, slices, and unfoldings (3D for intuition; general holds for any order)
For $\mathcal{X} \in \mathbb{R}^{I \times J \times K}$:

- **Mode-1 fiber:** $\mathcal{X}_{:,j,k} \in \mathbb{R}^I$ (column); **mode-2 fiber:** $\mathcal{X}_{i,:,k} \in \mathbb{R}^J$ (row); **mode-3 fiber:** $\mathcal{X}_{i,j,:} \in \mathbb{R}^K$ (tube).
- **Horizontal slice:** $\mathcal{X}_{i,:, :} \in \mathbb{R}^{J \times K}$; **lateral slice:** $\mathcal{X}_{:,j,:} \in \mathbb{R}^{I \times K}$; **frontal slice:** $\mathcal{X}_{:, :, k} \in \mathbb{R}^{I \times J}$.
- **Mode- n unfolding (general).** For $\mathcal{X} \in \mathbb{R}^{I_1 \times \dots \times I_N}$ and $n \in \{1, \dots, N\}$,

$$X_{(n)} \in \mathbb{R}^{I_n \times (\prod_{m \neq n} I_m)}$$

is formed by stacking all mode- n fibers as columns. Any consistent column-order of the remaining modes is acceptable; just use the same convention throughout an analysis.

Concrete 3D examples Let a 3D tensor be indexed $T_{i,j,k}$ with shape (I, J, K) .

1. **Grayscale video clip:** $V \in \mathbb{R}^{T \times H \times W}$, where $V_{t,y,x}$ is the pixel intensity at frame t , row y , column x .
2. **Color image:** $I \in \mathbb{R}^{H \times W \times 3}$ (channels last) or $I \in \mathbb{R}^{3 \times H \times W}$ (channels first). Entry $I_{y,x,c}$ with $c \in \{R, G, B\}$.
3. **Medical volume (MRI/CT):** $M \in \mathbb{R}^{X \times Y \times Z}$; $M_{x,y,z}$ is a voxel intensity.
4. **Sales data cube:** $S \in \mathbb{R}^{P \times S \times M}$ for (product, store, month); $S_{p,s,m}$ gives units sold.

Video tensors in practice

- **Single video (RGB):** shape $(T, H, W, 3)$ or $(3, T, H, W)$.

Shape abbreviations for video tensors We use the following axis names:

- N — **batch size:** number of videos in the mini-batch.
- T — **time:** number of frames per clip (e.g., $T \in \{8, 16, 32\}$).
- H — **height:** frame height in pixels (e.g., $H = 224$).
- W — **width:** frame width in pixels (e.g., $W = 224$).
- C — **channels:** color or feature channels (e.g., $C = 3$ for RGB, $C = 1$ for grayscale).

Thus a mini-batch of videos is stored as either

$$(N, T, H, W, C) \text{ (channels last)} \quad \text{or} \quad (N, C, T, H, W) \text{ (channels first)}.$$

Frame rates in practice (videos for ML)

- **Real-world fps.** Cinema: 24 fps; TV/broadcast: 25 (PAL), 29.97 $\approx$ 30 (NTSC), 60 for sports; web/streaming: 24/25/30/60; phones/cams: 30/60 (and 120/240 for slow motion).
- **Effective fps used in ML.** Pipelines often downsample to **8–30 fps** (e.g., decode at 30 and keep every 2nd frame $\Rightarrow$ 15 fps) to reduce redundancy and compute.
- **Clip length for training.** Short clips are standard: $T \in \{8, 16, 32, 64\}$ frames; longer models may use 96–128+.
- **Temporal coverage.** For effective frame rate f_{eff} and T frames,

$$\text{coverage (seconds)} = \frac{T}{f_{\text{eff}}}.$$

Examples: $T=16$ at $f_{\text{eff}}=16$ fps $\Rightarrow$ 1.0 s; $T=32$ at $f_{\text{eff}}=8$ fps $\Rightarrow$ 4.0 s.

- **Inference practice.** Use multi-clip sampling (several temporal crops per video) rather than feeding an entire video at once.

Tiny numeric illustration A toy tensor $\mathcal{T} \in \mathbb{R}^{2 \times 3 \times 2}$:

$$\mathcal{T}[:, :, 1] = \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{bmatrix}, \quad \mathcal{T}[:, :, 2] = \begin{bmatrix} 7 & 8 & 9 \\ 10 & 11 & 12 \end{bmatrix}.$$

Here $t_{2,3,1} = 6$ (row 2, column 3, slice 1).

Order vs. “rank”

- **Order N** : the number of modes (axes); a property of shape.
- **Matrix rank (for $N = 2$)**: dimension of the column space. For $N > 2$, several tensor “ranks” exist (e.g., CP rank, Tucker multilinear ranks); these differ from both order and matrix rank.

2 Tucker Decomposition

The Tucker decomposition is a form of higher-order principal component analysis (PCA). It decomposes a tensor into a **core tensor** multiplied by a matrix along each mode.

2.1 Definition

For an N -way tensor $\mathcal{X} \in \mathbb{R}^{I_1 \times I_2 \times \dots \times I_N}$, the Tucker decomposition is:

$$\mathcal{X} \approx \mathcal{G} \times_1 \mathbf{A}^{(1)} \times_2 \mathbf{A}^{(2)} \times_3 \dots \times_N \mathbf{A}^{(N)}.$$

Where:

- $\mathcal{G} \in \mathbb{R}^{R_1 \times R_2 \times \dots \times R_N}$ is the **core tensor**. It represents the interactions between the components of the different modes. The values $R_1, R_2, \dots, R_N$ are the number of components (or “rank”) for each mode.
- $\mathbf{A}^{(n)} \in \mathbb{R}^{I_n \times R_n}$ is the **factor matrix** for mode n . It can be thought of as the principal components for that mode.
- $\times_n$ is the **n -mode product**.

Element-wise Definition for a 3D Tensor: For a 3D tensor, the decomposition is:

$$\mathcal{X} \approx \mathcal{G} \times_1 \mathbf{A}^{(1)} \times_2 \mathbf{A}^{(2)} \times_3 \mathbf{A}^{(3)}$$

This can be written element-wise as:

$$x_{ijk} \approx \sum_{p=1}^{R_1} \sum_{q=1}^{R_2} \sum_{r=1}^{R_3} g_{pqr} \cdot a_{ip}^{(1)} \cdot a_{jq}^{(2)} \cdot a_{kr}^{(3)}.$$

2.2 How to Compute the Tucker Decomposition

The most common algorithm for computing the Tucker decomposition is the **Higher-Order Singular Value Decomposition (HOSVD)**, which is a multi-linear generalization of the matrix SVD.

Algorithm: HOSVD

1. **Input:** Tensor $\mathcal{X}$, and the desired multilinear rank $(R_1, R_2, \dots, R_N)$.
2. **For** each mode $n = 1, 2, \dots, N$:
 - Unfold the tensor to get the mode- n matricization $\mathbf{X}_{(n)}$.
 - Compute the (truncated) SVD of this matrix: $\mathbf{X}_{(n)} \approx \mathbf{U}_n \mathbf{S}_n \mathbf{V}_n^T$.
 - Set the factor matrix $\mathbf{A}^{(n)}$ to be the matrix of the first R_n left singular vectors of $\mathbf{X}_{(n)}$, i.e., $\mathbf{A}^{(n)} = \mathbf{U}_n(:, 1 : R_n)$.
3. **Compute the Core Tensor:** The core tensor is computed to “connect” all the factor matrices. It is given by projecting the original tensor onto the subspace spanned by the factor matrices.

$$\mathcal{G} = \mathcal{X} \times_1 \mathbf{A}^{(1)T} \times_2 \mathbf{A}^{(2)T} \times_3 \dots \times_N \mathbf{A}^{(N)T}$$

4. **Output:** The core tensor $\mathcal{G}$ and the factor matrices $\mathbf{A}^{(1)}, \mathbf{A}^{(2)}, \dots, \mathbf{A}^{(N)}$.

Key Points:

- **Truncation:** The choice of R_n is crucial. It is often chosen based on the singular values from the SVD in step 2, keeping only the most significant components.
- **The Core Tensor:** The core tensor is not, in general, diagonal, unlike in the matrix SVD. This makes the Tucker decomposition more flexible but also harder to interpret directly.
- **Alternating Least Squares (Tucker-ALS):** HOSVD provides a good initial solution, but it is not optimal in the least-squares sense. The Tucker-ALS algorithm can be used to iteratively refine the solution by minimizing the reconstruction error $\|\mathcal{X} - \mathcal{G} \times_1 \mathbf{A}^{(1)} \times_2 \dots \times_N \mathbf{A}^{(N)}\|^2$.

3 Mode- n Tensor–Matrix Multiplication

Let $\mathcal{X} \in \mathbb{R}^{I_1 \times I_2 \times \dots \times I_N}$ be an N th-order tensor and $\mathbf{A} \in \mathbb{R}^{J \times I_1}$ a matrix. The *mode-1 product* of $\mathcal{X}$ and $\mathbf{A}$ is denoted

$$\mathcal{Y} = \mathcal{X} \times_1 \mathbf{A},$$

and produces a tensor

$$\mathcal{Y} \in \mathbb{R}^{J \times I_2 \times \dots \times I_N}.$$

Definition (mode-1)

Each entry of $\mathcal{Y}$ is given by

$$\boxed{\mathcal{Y}_{j,i_2,\dots,i_N} = \sum_{i_1=1}^{I_1} \mathbf{A}_{j,i_1} \mathcal{X}_{i_1,i_2,\dots,i_N}.} \quad (1)$$

This operation multiplies all *mode-1 fibers* (vectors obtained by fixing all indices except the first) of $\mathcal{X}$ by the matrix $\mathbf{A}$.

Matrix Unfolding Form (mode-1)

If $\mathbf{X}_{(1)}$ denotes the *mode-1 unfolding* of $\mathcal{X}$,

$$\mathbf{X}_{(1)} \in \mathbb{R}^{I_1 \times (I_2 I_3 \dots I_N)},$$

then the mode-1 product can be expressed compactly as

$$\mathbf{Y}_{(1)} = \mathbf{A} \mathbf{X}_{(1)}, \quad \text{and } \mathcal{Y} = \text{fold}_1(\mathbf{Y}_{(1)}). \quad (2)$$

Example (mode-2)

Let $\mathcal{X} \in \mathbb{R}^{2 \times 3 \times 2}$ be the same tensor as above, and let

$$\mathcal{X}(:, :, 1) = \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{bmatrix}, \quad \mathcal{X}(:, :, 2) = \begin{bmatrix} 7 & 8 & 9 \\ 10 & 11 & 12 \end{bmatrix}, \quad \mathbf{B} = \begin{bmatrix} 1 & 0 & 1 \\ 0 & 1 & 1 \end{bmatrix} \in \mathbb{R}^{2 \times 3}.$$

Define $\mathcal{Z} = \mathcal{X} \times_2 \mathbf{B}$. Since $\mathbf{B} \in \mathbb{R}^{2 \times 3}$ acts on mode 2 (size 3), the result has size $2 \times 2 \times 2$. By the mode-2 product definition,

$$\mathcal{Z}_{i_1, :, i_3} = \mathbf{B} \mathcal{X}_{i_1, :, i_3}^\top \quad (\text{as a column}), \quad \text{equivalently } \mathcal{Z}_{i_1, j_2, i_3} = \sum_{s=1}^3 \mathbf{B}_{j_2, s} \mathcal{X}_{i_1, s, i_3}.$$

Examples.

$$\mathcal{X}_{1, :, 1} = \begin{bmatrix} 1 & 2 & 3 \end{bmatrix} \Rightarrow \mathcal{Z}_{1, :, 1} = \mathbf{B} \begin{bmatrix} 1 \\ 2 \\ 3 \end{bmatrix} = \begin{bmatrix} 1 \cdot 1 + 0 \cdot 2 + 1 \cdot 3 \\ 0 \cdot 1 + 1 \cdot 2 + 1 \cdot 3 \end{bmatrix} = \begin{bmatrix} 4 \\ 5 \end{bmatrix}.$$

$$\mathcal{X}_{2, :, 2} = \begin{bmatrix} 10 & 11 & 12 \end{bmatrix} \Rightarrow \mathcal{Z}_{2, :, 2} = \mathbf{B} \begin{bmatrix} 10 \\ 11 \\ 12 \end{bmatrix} = \begin{bmatrix} 10 + 12 \\ 11 + 12 \end{bmatrix} = \begin{bmatrix} 22 \\ 23 \end{bmatrix}.$$

Hence, each mode-2 fiber $\mathcal{X}_{i_1, :, i_3}$ (length 3) is linearly transformed by $\mathbf{B}$ to a length-2 vector, shrinking the second mode from 3 to 2.

General Mode- n Product

More generally, let $\mathcal{X} \in \mathbb{R}^{I_1 \times I_2 \times \dots \times I_N}$ and $\mathbf{A} \in \mathbb{R}^{J \times I_n}$. The *mode- n product* of $\mathcal{X}$ and $\mathbf{A}$ is denoted by

$$\mathcal{Y} = \mathcal{X} \times_n \mathbf{A},$$

and produces a tensor of size

$$\mathcal{Y} \in \mathbb{R}^{I_1 \times \dots \times I_{n-1} \times J \times I_{n+1} \times \dots \times I_N}.$$

Its entries are defined as

$$\boxed{\mathcal{Y}_{i_1, \dots, i_{n-1}, j, i_{n+1}, \dots, i_N} = \sum_{i_n=1}^{I_n} \mathbf{A}_{j, i_n} \mathcal{X}_{i_1, \dots, i_n, \dots, i_N}.} \quad (3)$$

Equivalently, in matricized form, if $\mathbf{X}_{(n)}$ denotes the mode- n unfolding of $\mathcal{X}$, then

$$\mathbf{Y}_{(n)} = \mathbf{A} \mathbf{X}_{(n)}, \quad \mathcal{Y} = \text{fold}_n(\mathbf{Y}_{(n)}). \quad (4)$$

This general definition reduces to the mode-1 product when $n = 1$ and is fundamental in multilinear algebra, particularly in the Tucker and PARAFAC tensor decompositions.

4 Examples of Tucker Decomposition

4.1 Example 1: Image Compression (3D Tensor - Color Image)

Consider a color image represented as a 3D tensor $\mathcal{I} \in \mathbb{R}^{H \times W \times 3}$, where H is height, W is width, and the 3 channels are Red, Green, and Blue.

1. **Goal:** Compress the image by finding a reduced representation.
2. **Decomposition:** We apply Tucker decomposition: $\mathcal{I} \approx \mathcal{G} \times_1 \mathbf{A} \times_2 \mathbf{B} \times_3 \mathbf{C}$.
 - $\mathbf{A} \in \mathbb{R}^{H \times R_1}$: Factor matrix for the “height” mode. Its columns can be seen as eigen-images along the vertical direction.
 - $\mathbf{B} \in \mathbb{R}^{W \times R_2}$: Factor matrix for the “width” mode. Its columns are eigen-images along the horizontal direction.
 - $\mathbf{C} \in \mathbb{R}^{3 \times R_3}$: Factor matrix for the “color” mode. Its columns represent dominant color combinations (e.g., one column might capture a “reddish” component, another a “bluish” one).
 - $\mathcal{G} \in \mathbb{R}^{R_1 \times R_2 \times R_3}$: The core tensor, governing how these spatial and color components interact.
3. **Compression:** By choosing $R_1 < H$, $R_2 < W$, and $R_3 \leq 3$, we achieve compression. We only need to store the core tensor $\mathcal{G}$ and the three factor matrices, which requires $R_1 R_2 R_3 + H R_1 + W R_2 + 3 R_3$ elements, which can be much less than the original $H \times W \times 3$ elements.

4.2 Example 2: Spatio-Temporal Data (3D Tensor)

Consider a dataset tracking the monthly temperature of 100 cities over 50 years, forming a tensor $\mathcal{T} \in \mathbb{R}^{100 \times 12 \times 50}$ (Cities $\times$ Months $\times$ Years).

1. **Goal:** Discover the underlying patterns across space, seasonal time, and long-term time.
2. **Decomposition:** $\mathcal{T} \approx \mathcal{G} \times_1 \mathbf{A} \times_2 \mathbf{B} \times_3 \mathbf{C}$.
 - $\mathbf{A} \in \mathbb{R}^{100 \times R_{\text{cities}}}$: Factor matrix for the “cities” mode. Each column represents a “cluster” or “archetype” of cities with similar temperature profiles (e.g., a “coastal warm” cluster, an “inland cold” cluster).
 - $\mathbf{B} \in \mathbb{R}^{12 \times R_{\text{months}}}$: Factor matrix for the “months” mode. Its columns represent fundamental seasonal patterns (e.g., one for summer-winter contrast, another for spring-fall).
 - $\mathbf{C} \in \mathbb{R}^{50 \times R_{\text{years}}}$: Factor matrix for the “years” mode. Its columns represent long-term temporal trends (e.g., a column for global warming, another for decadal oscillations like El Niño).
 - $\mathcal{G} \in \mathbb{R}^{R_{\text{cities}} \times R_{\text{months}} \times R_{\text{years}}}$: The core tensor. An entry $g_{p,q,r}$ tells us the strength of the interaction between the p -th city cluster, the q -th seasonal pattern, and the r -th long-term trend. A large value indicates a strong relationship.

This decomposition allows us to separate and independently analyze spatial, seasonal, and long-term trends in the complex dataset.

4.3 A numerical Example: Tucker decomposition of a $2 \times 3 \times 2$ tensor with a $2 \times 2 \times 2$ core

We illustrate a Tucker (HOSVD) decomposition for the third-order tensor $\mathcal{X} \in \mathbb{R}^{2 \times 3 \times 2}$ whose frontal slices are

$$\mathcal{X}(:, :, 1) = \begin{bmatrix} 1 & 3 & 5 \\ 2 & 4 & 6 \end{bmatrix}, \quad \mathcal{X}(:, :, 2) = \begin{bmatrix} 7 & 9 & 11 \\ 8 & 10 & 12 \end{bmatrix}.$$

Our goal is a Tucker model with multilinear rank $(R_1, R_2, R_3) = (2, 2, 2)$, i.e.,

$$\mathcal{X} = \mathcal{G} \times_1 \mathbf{U}_1 \times_2 \mathbf{U}_2 \times_3 \mathbf{U}_3, \quad \mathbf{U}_1 \in \mathbb{R}^{2 \times 2}, \mathbf{U}_2 \in \mathbb{R}^{3 \times 2}, \mathbf{U}_3 \in \mathbb{R}^{2 \times 2}, \mathcal{G} \in \mathbb{R}^{2 \times 2 \times 2}.$$

Mode- n unfoldings. Using the standard convention “bring mode- n to rows; flatten the remaining modes in column-major order,” the unfoldings are

$$\mathbf{X}_{(1)} = \begin{bmatrix} 1 & 3 & 5 & 7 & 9 & 11 \\ 2 & 4 & 6 & 8 & 10 & 12 \end{bmatrix} \in \mathbb{R}^{2 \times 6},$$

$$\mathbf{X}_{(2)} = \begin{bmatrix} 1 & 2 & 7 & 8 \\ 3 & 4 & 9 & 10 \\ 5 & 6 & 11 & 12 \end{bmatrix} \in \mathbb{R}^{3 \times 4},$$

$$\mathbf{X}_{(3)} = \begin{bmatrix} 1 & 2 & 3 & 4 & 5 & 6 \\ 7 & 8 & 9 & 10 & 11 & 12 \end{bmatrix} \in \mathbb{R}^{2 \times 6}.$$

HOSVD factors via SVDs. Compute the SVDs $\mathbf{X}_{(n)} = \mathbf{U}_n \mathbf{S}_n \mathbf{V}_n^\top$ and keep the first R_n left singular vectors. A numerically consistent choice (signs up to ± 1 are immaterial) is:

$$\mathbf{U}_1 \approx \begin{bmatrix} -0.663235 & -0.748411 \\ -0.748411 & 0.663235 \end{bmatrix}, \quad \mathbf{U}_2 \approx \begin{bmatrix} 0.421287 & 0.809846 \\ 0.565257 & 0.117551 \\ 0.709227 & -0.574744 \end{bmatrix},$$

and

$$\mathbf{U}_3 \approx \begin{bmatrix} -0.365212 & -0.930924 \\ -0.930924 & 0.365212 \end{bmatrix}.$$

(Here $\mathbf{U}_2$ is the matrix of the first two left singular vectors of $\mathbf{X}_{(2)}$; $\mathbf{U}_1$ and $\mathbf{U}_3$ are square and orthonormal.)

Core tensor. The (all-orthogonal) HOSVD core is obtained by

$$\mathcal{G} = \mathcal{X} \times_1 \mathbf{U}_1^\top \times_2 \mathbf{U}_2^\top \times_3 \mathbf{U}_3^\top \in \mathbb{R}^{2 \times 2 \times 2}.$$

Numerically (rounded to 6 decimals), the two frontal slices are

$$\mathcal{G}(:, :, 1) \approx \begin{bmatrix} 25.377662 & -0.005031 \\ -0.008304 & -2.308111 \end{bmatrix}, \quad \mathcal{G}(:, :, 2) \approx \begin{bmatrix} -0.025761 & -0.679877 \\ -0.324298 & -0.280592 \end{bmatrix}.$$

Reconstruction check. With the ranks $(2, 2, 2)$ there is no truncation, so the Tucker model reconstructs $\mathcal{X}$ (to floating-point precision):

$$\mathcal{X} = \mathcal{G} \times_1 \mathbf{U}_1 \times_2 \mathbf{U}_2 \times_3 \mathbf{U}_3.$$

Evaluating the right-hand side recovers the original two slices $\mathcal{X}(:, :, 1)$ and $\mathcal{X}(:, :, 2)$ with max absolute error $\lesssim 10^{-15}$.

Remarks on rank selection. The maximal multilinear ranks for a $2 \times 3 \times 2$ tensor are $(2, 3, 2)$. Using $(2, 2, 2)$ here illustrates a compact (square) core that still yields exact recovery for this specific $\mathcal{X}$. In applications, one typically selects (R_1, R_2, R_3) by energy thresholds on the singular values of $\mathbf{X}_{(n)}$ (e.g., 95%), scree plots, or cross-validation; smaller ranks compress but introduce approximation error.

Data and preprocessing note. In the datasets table elsewhere, replace “Diabet” by “Diabetes.” If you report outliers via Mahalanobis distance with a χ^2 cutoff, specify the covariance estimate (pooled vs. class-wise) and the chosen significance level (e.g., 0.01) for reproducibility.

5 Introduction

The **Canonical Polyadic (CP) decomposition**, also known as **CANDECOMP/PARAFAC**, is one of the most fundamental tensor factorization methods. It generalizes the matrix singular value decomposition (SVD) to higher-order tensors by decomposing a tensor into a sum of rank-one components.

6 Mathematical Definition

6.1 Basic Formulation

For an N -way tensor $\mathcal{X} \in \mathbb{R}^{I_1 \times I_2 \times \dots \times I_N}$, the CP decomposition is:

$$\mathcal{X} \approx \sum_{r=1}^R \lambda_r \cdot \mathbf{a}_r^{(1)} \circ \mathbf{a}_r^{(2)} \circ \dots \circ \mathbf{a}_r^{(N)}.$$

Where:

- R : **Rank** of the decomposition (number of components)
- $\lambda_r \in \mathbb{R}$: Weight for the r -th component
- $\mathbf{a}_r^{(n)} \in \mathbb{R}^{I_n}$: Factor vector for mode n and component r
- $\circ$: **Outer product** operator

6.2 Element-wise Formulation

For a 3-way tensor $\mathcal{X} \in \mathbb{R}^{I \times J \times K}$, the element-wise formulation is:

$$x_{ijk} \approx \sum_{r=1}^R \lambda_r \cdot a_{ir}^{(1)} \cdot a_{jr}^{(2)} \cdot a_{kr}^{(3)},$$

where $\mathbf{A}^{(1)} = [a_{ir}^{(1)}] \in \mathbb{R}^{I \times R}$, $\mathbf{A}^{(2)} = [a_{jr}^{(2)}] \in \mathbb{R}^{J \times R}$, and $\mathbf{A}^{(3)} = [a_{kr}^{(3)}] \in \mathbb{R}^{K \times R}$.

6.3 Matrix Formulation

Using factor matrices, the CP decomposition can be written as:

$$\mathcal{X} \approx \llbracket \lambda; \mathbf{A}^{(1)}, \mathbf{A}^{(2)}, \mathbf{A}^{(3)} \rrbracket,$$

where $\llbracket \cdot \rrbracket$ denotes the **Kruskal operator**.

7 Key Properties

7.1 Rank-One Components

Each term $\mathbf{a}_r^{(1)} \circ \mathbf{a}_r^{(2)} \circ \mathbf{a}_r^{(3)}$ is a **rank-one tensor**. For a 3-way tensor:

$$(\mathbf{a}_r^{(1)} \circ \mathbf{a}_r^{(2)} \circ \mathbf{a}_r^{(3)})_{ijk} = a_{ir}^{(1)} \cdot a_{jr}^{(2)} \cdot a_{kr}^{(3)}.$$

7.2 Uniqueness

CP decomposition often has **essential uniqueness** under mild conditions. This means the factor matrices are unique up to:

- **Permutation ambiguity:** Columns can be reordered
- **Scaling ambiguity:** Columns can be scaled (as long as the product across modes remains constant)

Formally, if $\llbracket \mathbf{A}^{(1)}, \mathbf{A}^{(2)}, \mathbf{A}^{(3)} \rrbracket = \llbracket \tilde{\mathbf{A}}^{(1)}, \tilde{\mathbf{A}}^{(2)}, \tilde{\mathbf{A}}^{(3)} \rrbracket$, then there exists a permutation matrix $\mathbf{P}$ and diagonal scaling matrices $\mathbf{\Lambda}_1, \mathbf{\Lambda}_2, \mathbf{\Lambda}_3$ such that:

$$\tilde{\mathbf{A}}^{(1)} = \mathbf{A}^{(1)} \mathbf{P} \mathbf{\Lambda}_1, \quad \tilde{\mathbf{A}}^{(2)} = \mathbf{A}^{(2)} \mathbf{P} \mathbf{\Lambda}_2, \quad \tilde{\mathbf{A}}^{(3)} = \mathbf{A}^{(3)} \mathbf{P} \mathbf{\Lambda}_3,$$

with $\mathbf{\Lambda}_1 \mathbf{\Lambda}_2 \mathbf{\Lambda}_3 = \mathbf{I}$.

8 Computational Framework

8.1 Optimization Problem

The CP decomposition is obtained by solving:

$$\min_{\mathbf{A}^{(1)}, \mathbf{A}^{(2)}, \mathbf{A}^{(3)}} \left\| \mathcal{X} - \llbracket \mathbf{A}^{(1)}, \mathbf{A}^{(2)}, \mathbf{A}^{(3)} \rrbracket \right\|_F^2.$$

8.2 Alternating Least Squares (ALS) Algorithm

The most popular method for computing CP decomposition is **CP-ALS**, which solves for each factor matrix alternately while keeping others fixed.

ALS Update Equations (3-way)

$$\begin{aligned}\mathbf{A}^{(1)} &= \mathbf{X}_{(1)} (\mathbf{A}^{(3)} \odot \mathbf{A}^{(2)}) \left(\mathbf{A}^{(3)\top} \mathbf{A}^{(3)} * \mathbf{A}^{(2)\top} \mathbf{A}^{(2)} \right)^\dagger, \\ \mathbf{A}^{(2)} &= \mathbf{X}_{(2)} (\mathbf{A}^{(3)} \odot \mathbf{A}^{(1)}) \left(\mathbf{A}^{(3)\top} \mathbf{A}^{(3)} * \mathbf{A}^{(1)\top} \mathbf{A}^{(1)} \right)^\dagger, \\ \mathbf{A}^{(3)} &= \mathbf{X}_{(3)} (\mathbf{A}^{(2)} \odot \mathbf{A}^{(1)}) \left(\mathbf{A}^{(2)\top} \mathbf{A}^{(2)} * \mathbf{A}^{(1)\top} \mathbf{A}^{(1)} \right)^\dagger.\end{aligned}$$

Key Operations Explained

- $\mathbf{X}_{(n)}$: Mode- n unfolding of tensor $\mathcal{X}$
- $\odot$: **Khatri-Rao product** (column-wise Kronecker product)
- $*$: **Hadamard product** (element-wise multiplication)
- $\dagger$: Moore-Penrose pseudoinverse

8.3 Khatri-Rao Product

For matrices $\mathbf{B} \in \mathbb{R}^{J \times R}$ and $\mathbf{C} \in \mathbb{R}^{K \times R}$, the Khatri-Rao product is:

$$\mathbf{C} \odot \mathbf{B} = [\mathbf{c}_1 \otimes \mathbf{b}_1 \mid \mathbf{c}_2 \otimes \mathbf{b}_2 \mid \cdots \mid \mathbf{c}_R \otimes \mathbf{b}_R] \in \mathbb{R}^{JK \times R}.$$

Element-wise:

$$(\mathbf{C} \odot \mathbf{B})_{(j-1)K+k,r} = b_{jr} \cdot c_{kr}.$$

9 CP-ALS Algorithm

10 Practical Considerations (Initialization, Rank, Normalization, Variants, and Implementation)

In practice, successful tensor factorizations hinge less on the abstract formulas and more on a handful of design choices that interact: how you initialize, how you pick ranks, how you normalize and scale factors, what constraints you impose, and how you monitor and implement the algorithm. This section collects those ingredients in one place, using the notation established above (e.g., $\mathcal{X}$, $\mathbf{A}^{(n)}$, $\mathbf{X}_{(n)}$, $\odot$, and $*$).

Algorithm 1 CP-ALS for 3-way Tensor

Require: Tensor $\mathcal{X} \in \mathbb{R}^{I \times J \times K}$, rank R , max iterations T , tolerance ϵ

Ensure: Factor matrices $\mathbf{A}^{(1)}, \mathbf{A}^{(2)}, \mathbf{A}^{(3)}$

```
1: Initialize  $\mathbf{A}^{(1)}, \mathbf{A}^{(2)}, \mathbf{A}^{(3)}$  randomly or via SVD
2:  $error \leftarrow \infty, prev\_error \leftarrow \infty$ 
3: for  $t = 1$  to  $T$  do
4:    $\mathbf{A}^{(1)} \leftarrow \mathbf{X}_{(1)}(\mathbf{A}^{(3)} \odot \mathbf{A}^{(2)}) \left( \mathbf{A}^{(3)\top} \mathbf{A}^{(3)} * \mathbf{A}^{(2)\top} \mathbf{A}^{(2)} \right)^\dagger$ 
5:    $\mathbf{A}^{(2)} \leftarrow \mathbf{X}_{(2)}(\mathbf{A}^{(3)} \odot \mathbf{A}^{(1)}) \left( \mathbf{A}^{(3)\top} \mathbf{A}^{(3)} * \mathbf{A}^{(1)\top} \mathbf{A}^{(1)} \right)^\dagger$ 
6:    $\mathbf{A}^{(3)} \leftarrow \mathbf{X}_{(3)}(\mathbf{A}^{(2)} \odot \mathbf{A}^{(1)}) \left( \mathbf{A}^{(2)\top} \mathbf{A}^{(2)} * \mathbf{A}^{(1)\top} \mathbf{A}^{(1)} \right)^\dagger$ 
7:    $prev\_error \leftarrow error$ 
8:    $error \leftarrow \|\mathcal{X} - \llbracket \mathbf{A}^{(1)}, \mathbf{A}^{(2)}, \mathbf{A}^{(3)} \rrbracket\|_F^2$ 
9:   if  $|error - prev\_error| < \epsilon$  then
10:    break
11:   end if
12: end for
13: return  $\mathbf{A}^{(1)}, \mathbf{A}^{(2)}, \mathbf{A}^{(3)}$ 
```

Initialization. There are four common starting points. Random initialization is the simplest and, with multiple restarts, often sufficient for moderate sizes; it injects diversity but may require careful stopping criteria to avoid needless iterations. SVD-based starts compute leading singular vectors of the mode- n unfoldings $\mathbf{X}_{(n)}$, then use those as columns of $\mathbf{A}^{(n)}$; this is fast and usually stabilizes early iterations. HOSVD-based starts go one step further by taking the truncated left singular subspaces of *each* unfolding $\mathbf{X}_{(n)}$ to form orthonormal $\mathbf{A}^{(n)}$, and then forming a core via projection; this is a robust one-shot choice that reduces the initial error without iteration. Finally, when an exact or nearly-exact low-rank structure is suspected (e.g., synthetic data), direct trilinear or problem-specific initializations can dramatically accelerate convergence by placing the iterates near a good basin from the outset.

Choosing ranks. Selecting the multilinear ranks for Tucker or the CP rank R is both statistical and computational. A pragmatic workflow is to hold out a subset of entries or slices and choose R (or $(R_1, \dots, R_N)$) that minimizes a predictive criterion on the hold-out set. In the absence of held-out data, one inspects the singular spectra of $\mathbf{X}_{(n)}$ and looks for a stable “elbow” (scree analysis) across modes; a compatible tuple $(R_1, \dots, R_N)$ preserves most of the mode-wise energy while avoiding over-parameterization. For CP, the *core consistency diagnostic* from chemometrics checks if the implied core is close to superdiagonal; large departures can indicate an overestimated rank. Bayesian formulations (automatic relevance determination, sparsity priors) or information criteria provide principled alternatives when computation permits.

Normalization and scaling. Because CP factors are only determined up to per-component scalings and permutations, it is advantageous to regularize scale during iterations. A common scheme is column normalization: after each ALS update, rescale each column of $\mathbf{A}^{(n)}$ to unit ℓ_2 norm and absorb the magnitude into component weights λ_r , so

that

$$\mathcal{X} \approx \sum_{r=1}^R \lambda_r \frac{\mathbf{a}_r^{(1)}}{\|\mathbf{a}_r^{(1)}\|} \circ \frac{\mathbf{a}_r^{(2)}}{\|\mathbf{a}_r^{(2)}\|} \circ \frac{\mathbf{a}_r^{(3)}}{\|\mathbf{a}_r^{(3)}\|}.$$

When factors have wildly different scales or the data are heteroscedastic, one may instead apply Frobenius normalization to whole factor matrices, provided the absorbed weights are tracked consistently. These strategies stabilize Gram matrices (e.g., $\mathbf{A}^{(k)\top} \mathbf{A}^{(k)}$) that appear in normal equations.

Extending formulas to N -way tensors. For an N -way CP model $\mathcal{X} \approx \llbracket \mathbf{A}^{(1)}, \dots, \mathbf{A}^{(N)} \rrbracket$, the ALS update for mode n uses the mode- n unfolding and the Khatri–Rao product of the remaining factors:

$$\mathbf{A}^{(n)} = \mathbf{X}_{(n)} \left(\bigodot_{k \neq n} \mathbf{A}^{(k)} \right) \left(\ast_{k \neq n} (\mathbf{A}^{(k)\top} \mathbf{A}^{(k)}) \right)^\dagger.$$

Here, $\bigodot$ stacks column-wise Kronecker products across all $k \neq n$, while the Hadamard product of Gram matrices couples components across modes. This identity encapsulates most practical CP-ALS implementations.

Strengths and caveats in one breath. CP is compelling because, under mild conditions, it is essentially unique: components correspond to interpretable latent factors, unlike Tucker’s rotation ambiguity. It is storage-efficient and extends naturally to missing-data settings. At the same time, the nonconvex objective admits local minima, rank selection is delicate, Khatri–Rao structures can be computationally heavy at scale, and convergence may slow down when Gram matrices are ill-conditioned. HOSVD and Tucker often serve as excellent initializers or as flexible models when interpretability is secondary to reconstruction fidelity.

Variants that matter. Weighted CP treats missing or unequal-reliability entries via a binary or real-valued weight tensor $\mathcal{W}$ and minimizes

$$\min_{\{\mathbf{A}^{(n)}\}} \|\mathcal{W} \ast (\mathcal{X} - \llbracket \mathbf{A}^{(1)}, \dots, \mathbf{A}^{(N)} \rrbracket)\|_F^2,$$

thereby restricting influence to observed or trusted entries. Nonnegative CP imposes $\mathbf{A}^{(n)} \geq 0$, which is natural for count or intensity data and yields parts-based factors (updated via NNLS or multiplicative rules). Sparse CP adds ℓ_1 penalties to encourage parsimonious loadings and reduce cross-talk between components. Domain constraints—smoothness (Tikhonov), orthogonality ($\mathbf{A}^{(n)\top} \mathbf{A}^{(n)} = \mathbf{I}$), or boundedness—can be folded into subproblems to encode prior structure.

Applications in context. In chemometrics, CP captures excitation–emission fluorescence as sums of outer products, enabling multi-way calibration; in neuroscience, it disentangles subjects, channels, and time into interpretable activity profiles; in computer

vision, it supports multiview and video analysis; and in data mining, it underlies recommender systems and multi-relational link prediction. Tucker, with its core $\mathcal{G}$, excels when interactions between latent factors across modes must be retained and visualized.

Implementation and monitoring without a forest of bullets. Large-scale runs benefit from *implicit* Khatri–Rao multiplication (apply $Z = \mathbf{X}_{(n)}(\odot_{k \neq n} \mathbf{A}^{(k)})$ without forming the big matrix), exploitation of sparsity, and randomized sketches of unfoldings for quick rank exploration. Numerical stability improves with small Tikhonov loadings on Gram matrices, QR-based solves in place of raw pseudoinverses, and consistent column rescaling. Convergence is best tracked by the reconstruction loss $\|\mathcal{X} - \llbracket \mathbf{A}^{(1)}, \dots, \mathbf{A}^{(N)} \rrbracket\|_F^2$, its relative change per iteration, and (when applicable) performance on held-out entries; gradient norms are informative when line-search or proximal variants are used. Finally, a compact comparison with Tucker can be summarized verbally: CP trades flexibility for identifiability and parsimony, while Tucker trades identifiability for expressive cores and often smoother optimization—choosing between them is less a matter of principle and more a matter of the scientific question and computational budget.

Appendix

11 Products Used in This Note (with Examples)

This section summarizes all multilinear and matrix products appearing in the notes, with consistent notation and small numeric examples. Throughout, vectors are columns by default; matrices use bold capitals (e.g., $\mathbf{A}$), and tensors use calligraphic capitals (e.g., $\mathcal{X}$). Your macros: Kronecker $\otimes$, Khatri–Rao $\odot$, Hadamard $*$, and mode- n product $\times_n$ are used consistently below.

11.1 Outer Product ($\circ$)

The outer product of vectors builds a rank-one tensor. For two vectors $\mathbf{a} \in \mathbb{R}^I$ and $\mathbf{b} \in \mathbb{R}^J$,

$$\mathbf{a} \circ \mathbf{b} \in \mathbb{R}^{I \times J}, \quad (\mathbf{a} \circ \mathbf{b})_{ij} = a_i b_j.$$

For three vectors $\mathbf{a} \in \mathbb{R}^I$, $\mathbf{b} \in \mathbb{R}^J$, $\mathbf{c} \in \mathbb{R}^K$,

$$\mathbf{a} \circ \mathbf{b} \circ \mathbf{c} \in \mathbb{R}^{I \times J \times K}, \quad (\mathbf{a} \circ \mathbf{b} \circ \mathbf{c})_{ijk} = a_i b_j c_k.$$

Example: Let $\mathbf{a} = \begin{bmatrix} 2 \\ 1 \end{bmatrix}$, $\mathbf{b} = \begin{bmatrix} 3 \\ -1 \\ 4 \end{bmatrix}$. Then

$$\mathbf{a} \circ \mathbf{b} = \begin{bmatrix} 2 \\ 1 \end{bmatrix} \begin{bmatrix} 3 & -1 & 4 \end{bmatrix} = \begin{bmatrix} 6 & -2 & 8 \\ 3 & -1 & 4 \end{bmatrix}.$$

11.2 Hadamard Product (*)

The Hadamard (element-wise) product between two matrices of the same size $A, B \in \mathbb{R}^{m \times n}$ is

$$A * B \in \mathbb{R}^{m \times n}, \quad (A * B)_{ij} = A_{ij}B_{ij}.$$

Example: With $\mathbf{A} = \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix}$, $\mathbf{B} = \begin{bmatrix} 5 & 6 \\ 7 & 8 \end{bmatrix}$,

$$\mathbf{A} * \mathbf{B} = \begin{bmatrix} 1 \cdot 5 & 2 \cdot 6 \\ 3 \cdot 7 & 4 \cdot 8 \end{bmatrix} = \begin{bmatrix} 5 & 12 \\ 21 & 32 \end{bmatrix}.$$

Note: In these notes, the macro `\hadamard` is mapped to `*`. If you prefer a symbol (e.g., $\otimes$ or $\odot$), just remap the macro.

11.3 Kronecker Product ($\otimes$)

For $\mathbf{A} \in \mathbb{R}^{m \times n}$ and $\mathbf{B} \in \mathbb{R}^{p \times q}$,

$$\mathbf{A} \otimes \mathbf{B} \in \mathbb{R}^{(mp) \times (nq)}$$

is the block matrix

$$\mathbf{A} \otimes \mathbf{B} = \begin{bmatrix} A_{11}\mathbf{B} & \cdots & A_{1n}\mathbf{B} \\ \vdots & \ddots & \vdots \\ A_{m1}\mathbf{B} & \cdots & A_{mn}\mathbf{B} \end{bmatrix}.$$

Example: With $\mathbf{A} = \begin{bmatrix} 1 & 2 \\ 0 & 1 \end{bmatrix}$, $\mathbf{B} = \begin{bmatrix} a & b \\ c & d \end{bmatrix}$,

$$\mathbf{A} \otimes \mathbf{B} = \begin{bmatrix} \mathbf{B} & 2\mathbf{B} \\ \mathbf{0} & \mathbf{B} \end{bmatrix} = \begin{bmatrix} a & b & 2a & 2b \\ c & d & 2c & 2d \\ 0 & 0 & a & b \\ 0 & 0 & c & d \end{bmatrix}.$$

11.4 Khatri–Rao Product ($\odot$)

The Khatri–Rao product is the *column-wise* Kronecker product. If $\mathbf{B} \in \mathbb{R}^{J \times R}$ and $\mathbf{C} \in \mathbb{R}^{K \times R}$ have the same number of columns,

$$\mathbf{C} \odot \mathbf{B} = [\mathbf{c}_1 \otimes \mathbf{b}_1 \mid \cdots \mid \mathbf{c}_R \otimes \mathbf{b}_R] \in \mathbb{R}^{(JK) \times R}.$$

Entry-wise:

$$(\mathbf{C} \odot \mathbf{B})_{(j-1)K+k, r} = b_{jr} c_{kr}.$$

Example: Let $\mathbf{B} = \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix}$, $\mathbf{C} = \begin{bmatrix} 5 & 6 \\ 7 & 8 \end{bmatrix}$. The first column is $\mathbf{c}_1 \otimes \mathbf{b}_1 = \begin{bmatrix} 5 \\ 7 \end{bmatrix} \otimes \begin{bmatrix} 1 \\ 3 \end{bmatrix} = \begin{bmatrix} 5 \\ 15 \\ 7 \\ 21 \end{bmatrix}$. The second column is $\begin{bmatrix} 6 \\ 8 \end{bmatrix} \otimes \begin{bmatrix} 2 \\ 4 \end{bmatrix} = \begin{bmatrix} 12 \\ 24 \\ 16 \\ 32 \end{bmatrix}$. Hence

$$\mathbf{C} \odot \mathbf{B} = \begin{bmatrix} 5 & 12 \\ 15 & 24 \\ 7 & 16 \\ 21 & 32 \end{bmatrix}.$$

11.5 Useful Identities and Relationships

For comfortable sizes:

$$\begin{aligned} (A \otimes B)(C \otimes D) &= (AC) \otimes (BD), \\ (A \otimes B)^\top &= A^\top \otimes B^\top, \\ (A * B)\mathbf{1} &= (A\mathbf{1}) * (B\mathbf{1}), \quad (\text{element-wise with matching sizes}) \\ \text{vec}(AXB^\top) &= (B \otimes A) \text{vec}(X), \\ \mathbf{Y}_{(n)} &= \mathbf{A} \mathbf{X}_{(n)} \iff \mathcal{Y} = \mathcal{X} \times_n \mathbf{A}, \\ (\mathbf{C} \odot \mathbf{B})^\top (\mathbf{C} \odot \mathbf{B}) &= (\mathbf{C}^\top \mathbf{C}) * (\mathbf{B}^\top \mathbf{B}). \end{aligned}$$

Example (Gram identity for Khatri–Rao): If $\mathbf{B}, \mathbf{C}$ share R columns, the $R \times R$ Gram matrix of $\mathbf{C} \odot \mathbf{B}$ is the Hadamard product of the Gram matrices:

$$\underbrace{(\mathbf{C} \odot \mathbf{B})^\top (\mathbf{C} \odot \mathbf{B})}_{\in \mathbb{R}^{R \times R}} = \underbrace{\mathbf{C}^\top \mathbf{C}}_{\in \mathbb{R}^{R \times R}} * \underbrace{\mathbf{B}^\top \mathbf{B}}_{\in \mathbb{R}^{R \times R}}.$$

This identity underlies the CP-ALS normal equations used in the updates in these notes.

11.6 Mode- n Tensor–Matrix Product ($\times_n$)

For $\mathcal{X} \in \mathbb{R}^{I_1 \times \cdots \times I_N}$ and $\mathbf{A} \in \mathbb{R}^{J \times I_n}$,

$$\mathcal{Y} = \mathcal{X} \times_n \mathbf{A} \in \mathbb{R}^{I_1 \times \cdots \times I_{n-1} \times J \times I_{n+1} \times \cdots \times I_N},$$

with entries

$$\mathcal{Y}_{i_1, \dots, i_{n-1}, j, i_{n+1}, \dots, i_N} = \sum_{i_n=1}^{I_n} \mathbf{A}_{j, i_n} \mathcal{X}_{i_1, \dots, i_n, \dots, i_N}.$$

In unfolding form (bring mode n to rows):

$$\mathbf{Y}_{(n)} = \mathbf{A} \mathbf{X}_{(n)}, \quad \mathcal{Y} = \text{fold}_n(\mathbf{Y}_{(n)}).$$

Example (mode-1): With $\mathcal{X} \in \mathbb{R}^{2 \times 3 \times 2}$ and $\mathbf{A} \in \mathbb{R}^{3 \times 2}$,

$$\mathcal{Y} = \mathcal{X} \times_1 \mathbf{A}, \quad \mathcal{Y}_{:,1,1} = \mathbf{A} \mathcal{X}_{:,1,1}.$$

If $\mathcal{X}_{:,1,1} = [1, 4]^\top$ and $\mathbf{A} = \begin{bmatrix} 1 & 0 \\ 0 & 1 \\ 1 & 1 \end{bmatrix}$, then $\mathcal{Y}_{:,1,1} = [1, 4, 5]^\top$.

11.7 Tensor–Vector Product (Special Case of $\times_n$)

Taking $\mathbf{A} = \mathbf{a}^\top \in \mathbb{R}^{1 \times I_n}$ in the mode- n product contracts mode n :

$$\mathcal{Z} = \mathcal{X} \times_n \mathbf{a}^\top \in \mathbb{R}^{I_1 \times \dots \times I_{n-1} \times I_{n+1} \times \dots \times I_N},$$

$$\mathcal{Z}_{i_1, \dots, i_{n-1}, i_{n+1}, \dots, i_N} = \sum_{i_n=1}^{I_n} a_{i_n} \mathcal{X}_{i_1, \dots, i_n, \dots, i_N}.$$

Example: If $\mathcal{X} \in \mathbb{R}^{2 \times 3}$ (a matrix) and $\mathbf{a} \in \mathbb{R}^2$, then $\mathcal{X} \times_1 \mathbf{a}^\top$ yields a row vector $= \mathbf{a}^\top \mathcal{X}$.

11.8 Kruskal Operator $\llbracket \cdot \rrbracket$ (CP Model)

Given factor matrices $\mathbf{A}^{(1)} \in \mathbb{R}^{I_1 \times R}, \dots, \mathbf{A}^{(N)} \in \mathbb{R}^{I_N \times R}$ and (optionally) weights λ_r ,

$$\llbracket \mathbf{A}^{(1)}, \dots, \mathbf{A}^{(N)} \rrbracket = \sum_{r=1}^R \lambda_r \mathbf{a}_r^{(1)} \circ \dots \circ \mathbf{a}_r^{(N)},$$

where $\mathbf{a}_r^{(n)}$ is the r -th column of $\mathbf{A}^{(n)}$ (set $\lambda_r = 1$ if weights are absorbed into factors).

Unfolding identity (3-way):

$$\mathbf{X}_{(1)} \approx \mathbf{A}^{(1)} (\mathbf{A}^{(3)} \odot \mathbf{A}^{(2)})^\top, \quad \mathbf{X}_{(2)} \approx \mathbf{A}^{(2)} (\mathbf{A}^{(3)} \odot \mathbf{A}^{(1)})^\top, \quad \mathbf{X}_{(3)} \approx \mathbf{A}^{(3)} (\mathbf{A}^{(2)} \odot \mathbf{A}^{(1)})^\top.$$

11.9 Folding (Inverse of Unfolding)

The operation $\text{fold}_n(\mathbf{Y}_{(n)})$ is the inverse of the mode- n unfolding (matricization). While unfolding rearranges tensor entries into a matrix $\mathbf{Y}_{(n)}$, *folding* reconstructs the original multidimensional array by reversing that reordering.

Definition. Let $\mathcal{Y} \in \mathbb{R}^{I_1 \times I_2 \times \dots \times I_N}$ be an N -way tensor, and let $\mathbf{Y}_{(n)}$ denote its mode- n unfolding:

$$\mathbf{Y}_{(n)} \in \mathbb{R}^{I_n \times (I_1 \dots I_{n-1} I_{n+1} \dots I_N)}.$$

Then the **folding** operation

$$\mathcal{Y} = \text{fold}_n(\mathbf{Y}_{(n)})$$

rearranges the elements of $\mathbf{Y}_{(n)}$ back into their original tensor indices according to the lexicographic ordering used in the unfolding.

Conceptually. If the (i_n, j) -th entry of $\mathbf{Y}_{(n)}$ corresponds to the tensor element $\mathcal{Y}_{i_1, i_2, \dots, i_N}$, where the column index j encodes all indices except i_n via

$$j = 1 + \sum_{\substack{k=1 \\ k \neq n}}^N (i_k - 1) J_k, \quad J_k = \prod_{\substack{m=1 \\ m \neq n}}^{k-1} I_m,$$

then folding simply reverses this mapping to recover each $(i_1, \dots, i_N)$ entry.

Example (3D tensor). Suppose $\mathcal{Y} \in \mathbb{R}^{2 \times 3 \times 2}$. Its mode-2 unfolding is

$$\mathbf{Y}_{(2)} \in \mathbb{R}^{3 \times (2 \cdot 2)} = \mathbb{R}^{3 \times 4},$$

whose columns correspond to the mode-2 fibers of $\mathcal{Y}$ ordered lexicographically by the other indices (i_1, i_3) . Then

$$\mathcal{Y} = \text{fold}_2(\mathbf{Y}_{(2)})$$

means that column j of $\mathbf{Y}_{(2)}$ becomes the (i_1, i_3) -th lateral slice $\mathcal{Y}_{i_1, :, i_3}$. Folding thus reassembles the tensor by placing each column of $\mathbf{Y}_{(2)}$ back into its proper location in the $I_1 \times I_2 \times I_3$ grid.

Relationship to mode- n product. The folding operator naturally appears in definitions such as

$$\mathbf{Y}_{(n)} = \mathbf{A} \mathbf{X}_{(n)} \iff \mathcal{Y} = \text{fold}_n(\mathbf{Y}_{(n)}),$$

ensuring that the result of a matrix–tensor multiplication along mode n is represented again as an N -way tensor with its original shape (except that I_n is replaced by the number of rows of $\mathbf{A}$).

Computational note. In practice, `fold` (or `tensorize`) is implemented as a reshape or permutation of array indices, inverse to the `unfold` function. No arithmetic is performed—it is purely an indexing operation that restores the original multidimensional structure.

12 Spectral Decomposition and the Singular Value Decomposition (SVD)

This section recalls the spectral decomposition for symmetric matrices and then uses it to derive and interpret the SVD for general (possibly rectangular) matrices. We keep the notation consistent with the rest of the notes: bold capitals for matrices (e.g., $\mathbf{A}$), and $\mathbf{U}, \mathbf{S}, \mathbf{V}$ for SVD factors.

12.1 Spectral Decomposition (Real Symmetric Case)

Let $\mathbf{M} \in \mathbb{R}^{n \times n}$ be *symmetric*, i.e., $\mathbf{M} = \mathbf{M}^\top$. Then the **spectral theorem** states:

$$\mathbf{M} = \mathbf{Q} \mathbf{\Lambda} \mathbf{Q}^\top,$$

where

- $\mathbf{Q} \in \mathbb{R}^{n \times n}$ is **orthogonal** ($\mathbf{Q}^\top \mathbf{Q} = \mathbf{I}$), whose columns $\{\mathbf{q}_1, \dots, \mathbf{q}_n\}$ are orthonormal eigenvectors of $\mathbf{M}$;
- $\mathbf{\Lambda} = \text{diag}(\lambda_1, \dots, \lambda_n)$ contains the (real) **eigenvalues** of $\mathbf{M}$.

Interpretation. The symmetric matrix $\mathbf{M}$ acts as a scaling by λ_i along the orthonormal directions $\mathbf{q}_i$.

Properties.

- If $\mathbf{M}$ is **positive semidefinite** (PSD), then $\lambda_i \geq 0$ and we can write $\mathbf{M}^{1/2} = \mathbf{Q} \text{diag}(\sqrt{\lambda_i}) \mathbf{Q}^\top$.
- If $\mathbf{M}$ is **orthogonally diagonalizable** (i.e., symmetric), its eigenvectors form an orthonormal basis.

Example (spectral decomposition). Let

$$\mathbf{M} = \begin{bmatrix} 2 & 1 \\ 1 & 2 \end{bmatrix}.$$

The characteristic polynomial is $(2 - \lambda)^2 - 1 = \lambda^2 - 4\lambda + 3 = (\lambda - 3)(\lambda - 1)$, so eigenvalues are $\lambda_1 = 3$, $\lambda_2 = 1$ with corresponding orthonormal eigenvectors

$$\mathbf{q}_1 = \frac{1}{\sqrt{2}} \begin{bmatrix} 1 \\ 1 \end{bmatrix}, \quad \mathbf{q}_2 = \frac{1}{\sqrt{2}} \begin{bmatrix} 1 \\ -1 \end{bmatrix}.$$

Hence

$$\mathbf{Q} = \frac{1}{\sqrt{2}} \begin{bmatrix} 1 & 1 \\ 1 & -1 \end{bmatrix}, \quad \mathbf{\Lambda} = \begin{bmatrix} 3 & 0 \\ 0 & 1 \end{bmatrix}, \quad \mathbf{M} = \mathbf{Q} \mathbf{\Lambda} \mathbf{Q}^\top.$$

12.2 SVD via Spectral Decomposition

For a general $\mathbf{A} \in \mathbb{R}^{m \times n}$, form the *Gram* matrices

$$\mathbf{A}^\top \mathbf{A} \in \mathbb{R}^{n \times n} \quad \text{and} \quad \mathbf{A} \mathbf{A}^\top \in \mathbb{R}^{m \times m}.$$

Both are symmetric and PSD, so each admits a spectral decomposition:

$$\mathbf{A}^\top \mathbf{A} = \mathbf{V} \text{diag}(\sigma_1^2, \dots, \sigma_r^2, 0, \dots, 0) \mathbf{V}^\top, \quad \mathbf{A} \mathbf{A}^\top = \mathbf{U} \text{diag}(\sigma_1^2, \dots, \sigma_r^2, 0, \dots, 0) \mathbf{U}^\top,$$

where $r = \text{rank}(\mathbf{A})$ and $\sigma_1 \geq \dots \geq \sigma_r > 0$ are the **singular values** of $\mathbf{A}$.

Singular Value Decomposition (SVD). There exist orthogonal matrices $\mathbf{U} \in \mathbb{R}^{m \times m}$, $\mathbf{V} \in \mathbb{R}^{n \times n}$ and a rectangular diagonal matrix $\mathbf{S} \in \mathbb{R}^{m \times n}$ with nonnegative diagonal entries (the singular values) such that

$$\mathbf{A} = \mathbf{U} \mathbf{S} \mathbf{V}^\top.$$

If $m \geq n$,

$$\mathbf{S} = \begin{bmatrix} \text{diag}(\sigma_1, \dots, \sigma_r) & \mathbf{0} \\ \mathbf{0} & \mathbf{0} \end{bmatrix} \in \mathbb{R}^{m \times n};$$

if $m < n$, the nonzero diagonal block is in the top left of size $m \times m$.

Construction from spectra. If $\mathbf{V} = [\mathbf{v}_1, \dots, \mathbf{v}_n]$ diagonalizes $\mathbf{A}^\top \mathbf{A}$, define

$$\mathbf{u}_i = \frac{1}{\sigma_i} \mathbf{A} \mathbf{v}_i \quad (i = 1, \dots, r), \quad \sigma_i = \sqrt{\mathbf{v}_i^\top (\mathbf{A}^\top \mathbf{A}) \mathbf{v}_i}.$$

Then $\mathbf{U} = [\mathbf{u}_1, \dots, \mathbf{u}_m]$ (completed to orthonormal basis) and $\mathbf{V}$ yield $\mathbf{A} = \mathbf{U} \mathbf{S} \mathbf{V}^\top$.

Geometric interpretation. $\mathbf{A}$ maps the orthonormal basis $\{\mathbf{v}_i\}$ to orthogonal vectors $\{\sigma_i \mathbf{u}_i\}$:

$$\mathbf{A} \mathbf{v}_i = \sigma_i \mathbf{u}_i \quad (i = 1, \dots, r), \quad \mathbf{A} \mathbf{v}_i = \mathbf{0} \quad (i > r).$$

Key properties.

- **Unitary invariance:** $\|\mathbf{A}\|_2 = \sigma_{\max}$, $\|\mathbf{A}\|_F^2 = \sum_i \sigma_i^2$.
- **Best rank- k approximation (Eckart–Young):** If $\mathbf{S} = \text{diag}(\sigma_1, \dots)$, then

$$\mathbf{A}_k := \sum_{i=1}^k \sigma_i \mathbf{u}_i \mathbf{v}_i^\top \quad \text{minimizes} \quad \|\mathbf{A} - \mathbf{B}\|_F \quad \text{over all} \quad \text{rank}(\mathbf{B}) \leq k.$$

- **Relation to spectral decomposition:** $\mathbf{A}^\top \mathbf{A} = \mathbf{V} \mathbf{S}^\top \mathbf{S} \mathbf{V}^\top$ and $\mathbf{A} \mathbf{A}^\top = \mathbf{U} \mathbf{S} \mathbf{S}^\top \mathbf{U}^\top$ are spectral decompositions with eigenvalues σ_i^2 .

12.3 Examples (SVD)

Example 1: Diagonal matrix (trivial SVD). Let

$$\mathbf{A} = \begin{bmatrix} 3 & 0 \\ 0 & 2 \end{bmatrix}.$$

Then $\mathbf{A}^\top \mathbf{A} = \text{diag}(9, 4)$, so $\sigma_1 = 3$, $\sigma_2 = 2$. One SVD is

$$\mathbf{U} = \mathbf{I}_2, \quad \mathbf{V} = \mathbf{I}_2, \quad \mathbf{S} = \text{diag}(3, 2),$$

and the best rank-1 approximation is $\mathbf{A}_1 = 3 \mathbf{e}_1 \mathbf{e}_1^\top = \begin{bmatrix} 3 & 0 \\ 0 & 0 \end{bmatrix}$.

Example 2: Rank-1, non-symmetric matrix. Let

$$\mathbf{A} = \begin{bmatrix} 1 & 2 \\ 0 & 0 \end{bmatrix} \in \mathbb{R}^{2 \times 2}.$$

Compute

$$\mathbf{A}^\top \mathbf{A} = \begin{bmatrix} 1 & 2 \\ 2 & 4 \end{bmatrix} \Rightarrow \text{eigenvalues } \{5, 0\}, \text{ eigenvectors } \mathbf{v}_1 \propto \begin{bmatrix} 1 \\ 2 \end{bmatrix}, \mathbf{v}_2 \propto \begin{bmatrix} -2 \\ 1 \end{bmatrix}.$$

Normalize

$$\mathbf{v}_1 = \frac{1}{\sqrt{5}} \begin{bmatrix} 1 \\ 2 \end{bmatrix}, \quad \mathbf{v}_2 = \frac{1}{\sqrt{5}} \begin{bmatrix} -2 \\ 1 \end{bmatrix}, \quad \sigma_1 = \sqrt{5}, \sigma_2 = 0.$$

Then

$$\mathbf{u}_1 = \frac{1}{\sigma_1} \mathbf{A} \mathbf{v}_1 = \frac{1}{\sqrt{5}} \mathbf{A} \left(\frac{1}{\sqrt{5}} \begin{bmatrix} 1 \\ 2 \end{bmatrix} \right) = \frac{1}{\sqrt{5}} \cdot \frac{1}{\sqrt{5}} \begin{bmatrix} 5 \\ 0 \end{bmatrix} = \begin{bmatrix} 1 \\ 0 \end{bmatrix}.$$

Choose $\mathbf{u}_2 = \begin{bmatrix} 0 \\ 1 \end{bmatrix}$ to complete $\mathbf{U} = [\mathbf{u}_1 \ \mathbf{u}_2]$ orthonormally. Hence an SVD is

$$\mathbf{U} = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}, \quad \mathbf{S} = \begin{bmatrix} \sqrt{5} & 0 \\ 0 & 0 \end{bmatrix}, \quad \mathbf{V} = \begin{bmatrix} \frac{1}{\sqrt{5}} & -\frac{2}{\sqrt{5}} \\ \frac{2}{\sqrt{5}} & \frac{1}{\sqrt{5}} \end{bmatrix}, \quad \mathbf{A} = \mathbf{U} \mathbf{S} \mathbf{V}^\top.$$

Geometrically, $\mathbf{A}$ maps $\mathbf{v}_1$ to $\sqrt{5} \mathbf{u}_1$ and annihilates $\mathbf{v}_2$.

Example 3 (optional): Involutory orthogonal. Let $\mathbf{A} = \begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix}$. Then $\mathbf{A}^\top \mathbf{A} = \mathbf{I}$, so $\sigma_1 = \sigma_2 = 1$ and an SVD is

$$\mathbf{U} = \mathbf{A}, \quad \mathbf{S} = \mathbf{I}, \quad \mathbf{V} = \mathbf{I}.$$

$\mathbf{A}$ is a reflection/swapping operator: it rotates basis axes and preserves lengths.

13 Notes on the Moore–Penrose Pseudoinverse

What it is For any real matrix $A \in \mathbb{R}^{m \times n}$, the *Moore–Penrose pseudoinverse* $A^\dagger \in \mathbb{R}^{n \times m}$ is the unique matrix that satisfies the four Penrose conditions:

$$AA^\dagger A = A, \quad A^\dagger AA^\dagger = A^\dagger, \quad (AA^\dagger)^\top = AA^\dagger, \quad (A^\dagger A)^\top = A^\dagger A.$$

It generalizes the usual inverse to rectangular and/or rank-deficient matrices.

How to compute (SVD recipe) Let $A = U \Sigma V^\top$ be a singular value decomposition, with $\Sigma = \text{diag}(\sigma_1, \dots, \sigma_r, 0, \dots)$ and $r = \text{rank}(A)$. Then

$$A^\dagger = V \Sigma^\dagger U^\top, \quad \Sigma^\dagger = \text{diag}\left(\frac{1}{\sigma_1}, \dots, \frac{1}{\sigma_r}, 0, \dots\right).$$

Numerics. In practice, reciprocate only singular values above a tolerance (e.g., $\sigma_i > \tau$) to stabilize the inverse; this yields a truncated SVD pseudoinverse.

Useful special cases If A has full column rank ($m \geq n$, $\text{rank}(A) = n$):

$$A^\dagger = (A^\top A)^{-1} A^\top.$$

If A has full row rank ($m \leq n$, $\text{rank}(A) = m$):

$$A^\dagger = A^\top (AA^\top)^{-1}.$$

Least-squares interpretation For any $b \in \mathbb{R}^m$, the vector

$$x^\star = A^\dagger b$$

is the *minimum-norm* solution of the least-squares problem $\min_x \|Ax - b\|_2$. If the system $Ax = b$ is consistent, $x^\star$ is the minimum-norm exact solution.

Projectors and geometry

$P_{\text{col}} := AA^\dagger$ is the orthogonal projector onto $\text{col}(A)$, $P_{\text{row}} := A^\dagger A$ projects onto $\text{row}(A)$.

Both P_{col} and P_{row} are symmetric idempotent: $P^\top = P$, $P^2 = P$.

Handy identities (with caveats)

$$(A^\top)^\dagger = (A^\dagger)^\top, \quad (A^\dagger)^\dagger = A.$$

In general $(AB)^\dagger \neq B^\dagger A^\dagger$; equality needs extra rank/orthogonality assumptions.

Small worked example (tall, full column rank) Let

$$A = \begin{bmatrix} 1 & 2 \\ 3 & 4 \\ 5 & 6 \end{bmatrix} \in \mathbb{R}^{3 \times 2}.$$

Then $A^\top A = \begin{bmatrix} 35 & 44 \\ 44 & 56 \end{bmatrix}$ with inverse

$$(A^\top A)^{-1} = \frac{1}{24} \begin{bmatrix} 56 & -44 \\ -44 & 35 \end{bmatrix}.$$

Thus

$$A^\dagger = (A^\top A)^{-1} A^\top = \frac{1}{24} \begin{bmatrix} 56 & -44 \\ -44 & 35 \end{bmatrix} \begin{bmatrix} 1 & 3 & 5 \\ 2 & 4 & 6 \end{bmatrix} = \begin{bmatrix} -\frac{4}{3} & -\frac{1}{3} & \frac{2}{3} \\ \frac{13}{12} & \frac{1}{3} & -\frac{5}{12} \end{bmatrix}.$$

Solve least squares for $b = \begin{bmatrix} 1 \\ 0 \\ 1 \end{bmatrix}$:

$$x^\star = A^\dagger b = \begin{bmatrix} -\frac{2}{3} \\ \frac{2}{3} \end{bmatrix}, \quad Ax^\star = \begin{bmatrix} 2/3 \\ 2/3 \\ 2/3 \end{bmatrix}, \quad r = b - Ax^\star = \begin{bmatrix} 1/3 \\ -2/3 \\ 1/3 \end{bmatrix}.$$

Here $x^\star$ minimizes $\|Ax - b\|_2$ and has the smallest Euclidean norm among all minimizers.

Rank-deficient / ill-posed notes When A is ill-conditioned, a *damped* (Tikhonov) variant is often used:

$$A_{\lambda}^{\dagger} := (A^{\top} A + \lambda I)^{-1} A^{\top},$$

which corresponds to ridge-regularized least squares and is numerically more stable than a raw pseudoinverse for small singular values.